SMART BEINGS · WOOHOO PLATFORM

UI / UX
Integration Guide

Every state the interface must handle, what you can build today, and the contract for the staff screens that are still to come.

Guest surfaces | Device screen | Staff & admin | Accessibility

FOR	GUEST SURFACE	STAFF SURFACE
UI / UX engineers & designers	Buildable today	Contract only

READ FIRST

What you can build today

There are three interfaces in this product, and they are in **very different states of readiness**. Knowing which you are building determines whether you can start this week.

Interface	What it is	Status	Talks to
Device screen	The display on the in-room hub. Server-driven content, rendered by the Android app.	READY	The Android app renders `screen` messages — you specify how
Guest surface	A web chat, an in-room tablet UI, a demo console — anything where a *guest* asks for something.	READY	`POST /v1/turn` — three fields, works today
Staff & admin	Housekeeping queues, maintenance tickets, property configuration, escalation handling.	BLOCKED	**No API exists.** Part V is the proposed contract to design against

IF YOU ARE BUILDING STAFF SCREENS, READ THIS BEFORE PLANNING YOUR SPRINT

There is **no staff API and no authentication**. The service currently exposes exactly three things: a WebSocket for the device, `POST /v1/turn`, and `GET /health`. All device-facing.

The logic behind the staff screens exists and is tested — listing and assigning requests, acknowledging escalations, searching what the assistant did, property configuration, kill switches — but none of it is reachable over HTTP. Part V specifies the contract so you can design and mock against it now; building it is roughly a week once two decisions are made (§24).

The thing that shapes every screen you will design

THE ASSISTANT PROPOSES; THE PLATFORM DECIDES

The AI never acts on its own. It suggests an action, and a separate component authorises it. That means **a request appearing in a response does not mean it happened**, and your UI must reflect the difference.

Every action comes back with a *verdict*: allowed, refused, needs confirmation, or escalated to a human. A refusal is a **normal outcome**, not an error state — design it as information, not as a failure.

Contents

Part I — The guest surface

1 The one call you need
2 Response contract
3 The state machine
4 Worked examples

Part II — States you must handle

5 Conversation states
6 Confirmation — the critical one
7 Escalation — the other critical one
8 Refusals and limits
9 Degraded and offline

Part III — The device screen

10 How it works
11 Screen states
12 Latency and the filled pause

Part IV — Craft

13 Accessibility
14 Copy and tone
15 What never to show

Part V — Staff & admin

16 Screens needed
17 Proposed API contract
18 Live updates
19 Permissions model

Part VI — Practical

20 Running it locally
21 Test scenarios
22 Design checklist
23 Reference data
24 Decisions needed from you

PART I

The guest surface

1. The one call you need

```
POST http://localhost:8443/v1/turn
content-type: application/json

{ "deviceId": "d_511a", "transcript": "close the curtains and make it warmer" }
```

```
async function sendTurn(deviceId, transcript, asrConfidence = 0.95) {
  const res = await fetch('/v1/turn', {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ deviceId, transcript, asrConfidence }),
  });
  return res.json();
}
```

Three fields. Note what is **absent**: no room number, no property, no auth token. Identity comes from `deviceId` and the service derives everything else. There is nowhere in this API to specify a room, deliberately — which is also why a guest cannot ask about someone else's.

2. Response contract

```
{
  "turnId": "turn_ms6e6b0a0",
  "speech": "Curtains closed. I've set it to 22.5 degrees.",
  "screen": null,
  "lane": "agentic",
  "escalated": false,
  "awaitingConfirmation": false,
  "degradationLevel": 0,
  "totalMs": 52,
  "calls": [
    { "tool": "curtains.set",            "tier": "T1", "verdict": "ALLOW", "ok": true },
    { "tool": "climate.set_temperature", "tier": "T1", "verdict": "ALLOW", "ok": true }
  ]
}
```

Field	Type	What the UI does with it
`speech`	string	**The answer.** Display it, speak it, or both. Always short — capped around 45 words.
`screen`	string?	Present only when the display must carry something the words don't — an order total, an emergency number. **Render it whenever present.** May contain `\n`.
`awaitingConfirmation`	boolean	**A modal state, not a label.** See §6. Getting this wrong charges guests for things they didn't confirm.
`escalated`	boolean	A human was paged. Surface prominently and persistently. See §7.
`degradationLevel`	0–5	Non-zero means reduced capability. See §9.
`lane`	string	Diagnostic. `fast` / `agentic` / `clarify` / `confirm` / `degraded`. Useful in a debug view; not for guests.
`calls[]`	array	What was attempted and what policy said. Drives "what changed" affordances.
`totalMs`	number	Server-side duration. Debug views only.

2.1 Reading `calls[]` correctly

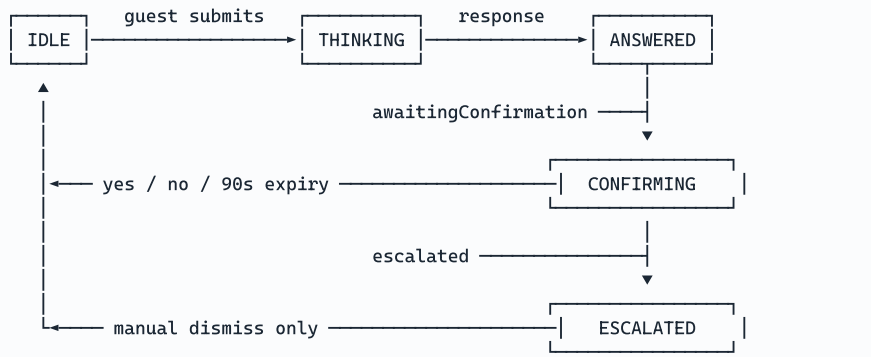
> **`VERDICT` AND `OK` ARE TWO DIFFERENT QUESTIONS**
>
> `verdict` asks *was it allowed?* `ok` asks *did it work?*
>
> A call can be `ALLOW` with `ok: false` — policy permitted it and the room system then failed. That is a genuinely different message to the guest than a refusal, and your UI should not collapse the two.

verdict	Means	UI treatment
`ALLOW`	Authorised, and executed if `ok`	Normal success. Show what changed.
`CONFIRM`	Frozen, waiting for a spoken yes	Enter confirmation state (§6)
`DENY`	Policy refused it	**Not an error.** Neutral, informative. No red, no alarm icon.
`ESCALATE`	Handed to a human	Prominent, persistent (§7)

2.2 HTTP status codes

Code	Meaning	UI
200	Turn completed — **including refusals**	Render `speech`. Do not treat as failure.
400	Missing field or bad JSON	Client bug
404	Unknown or revoked device	Configuration problem, not a guest problem
413	Body over 16 KB	Cap input length client-side
500	Internal fault	Honest error; offer the front desk. **Do not auto-retry** — it may have half-executed.

3. The state machine



Cross-cutting: DEGRADED (degradationLevel ≥ 2) · OFFLINE (network) · ERROR (5xx)

> **TWO STATES HAVE HARD RULES ATTACHED**
>
> **CONFIRMING** must not auto-advance, and **ESCALATED** must not auto-dismiss. Everything else is your call.

4. Worked examples

All captured from the running service.

4.1 Simple success

```
POST { "deviceId":"d_412a", "transcript":"turn on the lights" }

{ "speech":"Done.", "lane":"fast", "totalMs":39,
  "calls":[{ "tool":"lighting.set","tier":"T1","verdict":"ALLOW","ok":true }] }
```

39 ms because it never touched the AI — common requests are handled deterministically. Your loading state may barely appear; design for that.

4.2 Two things at once

```
POST { "transcript":"close the curtains and make it warmer" }

{ "speech":"Curtains closed. I've set it to 22.5 degrees.",
  "calls":[ curtains.set ALLOW ok, climate.set_temperature ALLOW ok ] }
```

One sentence, two actions, one reply. If you show “what changed” chips, this produces two.

4.3 A refusal — design this as information

```
POST { "transcript":"set the temperature to 35 degrees" }

{ "speech":"I couldn't do that just now. Would you like me to connect you to
            the front desk?",
  "calls":[{ "tool":"climate.set_temperature","verdict":"DENY",
             "ruleId":"R6.out_of_bounds","ok":false }] }
```

HTTP 200. The hotel permits 18–26 °C. Nothing is broken — do not show an error chrome.

4.4 Something that costs money

```
POST { "transcript":"I'd like a club sandwich and a beer" }

{ "speech":"I have 2 items ready to order. That's 1240 INR, charged to your room.
           Shall I place it?",
  "screen":"I have 2 items ready to order. That's 1240 INR, charged to your room.",
  "awaitingConfirmation": true,
  "calls":[{ "tool":"fnb.place_order","tier":"T3","verdict":"CONFIRM","ok":false }] }
```

Nothing has been ordered. `ok:false` is correct here — it was frozen, not executed.

4.5 The confirming turn

```
POST { "transcript":"yes" }

{ "speech":"Ordered — about 30 minutes. 1240 INR has gone to your room bill.",
  "lane":"confirm", "awaitingConfirmation": false,
  "calls":[{ "tool":"fnb.place_order","verdict":"ALLOW",
             "ruleId":"R11.confirmation_redeemed","ok":true }] }
```

4.6 An emergency

```
POST { "transcript":"I'm having chest pain" }

{ "speech":"I'm alerting the hotel team right now. If this is an emergency please
           call 112 — I'm not able to call them for you. I'll stay here with you.",
  "screen":"Emergency: 112\nHotel team alerted",
  "escalated": true,
  "calls":[{ "tool":"staff.escalate_urgent","tier":"T4","verdict":"ESCALATE" }] }
```

PART II

States you must handle

5. Conversation states

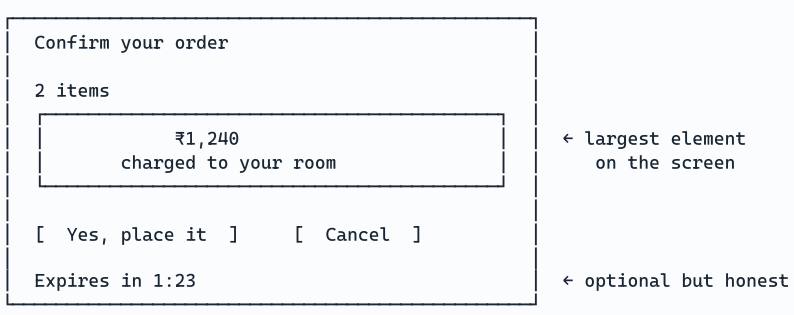
State	Detect by	Show	Notes
Thinking	Request in flight	Activity indicator	May last 30 ms or 2 s. Avoid a heavy animation that flashes.
Answered	Response, no flags	`speech`, plus `screen` if present	Optionally chips for what changed
Confirming	`awaitingConfirmation`	Total, prominent + yes/no	§6 — strict rules
Escalated	`escalated`	Persistent banner, emergency number	§7 — strict rules
Clarifying	`lane === "clarify"`	The question, invite a retry	Empty `calls`. Nothing was done.
Degraded	`degradationLevel ≥ 2`	Subtle notice of reduced capability	§9
Offline	Network failure	What still works	§9

6. Confirmation — the critical one

THREE RULES, AND THEY ARE NOT NEGOTIABLE

1. **Never auto-confirm.** Not on timeout, not on ambiguity, not on a "seems fine" heuristic. Only an explicit affirmative from the guest.
2. **Always show the total.** The guest is agreeing to a charge on their room bill. The screen is the record they can read and re-read; the spoken sentence is not.
3. **Always offer a way out.** A visible cancel, not just "say no".

SUGGESTED LAYOUT — CONFIRMATION



6.1 The 90-second expiry

The frozen action expires server-side after 90 seconds regardless of what your UI does. A late "yes" is therefore safe — the guest is told it expired and nothing is charged. Mirror the expiry in your UI so the two do not disagree, and clear the state on expiry rather than leaving a stale confirm button.

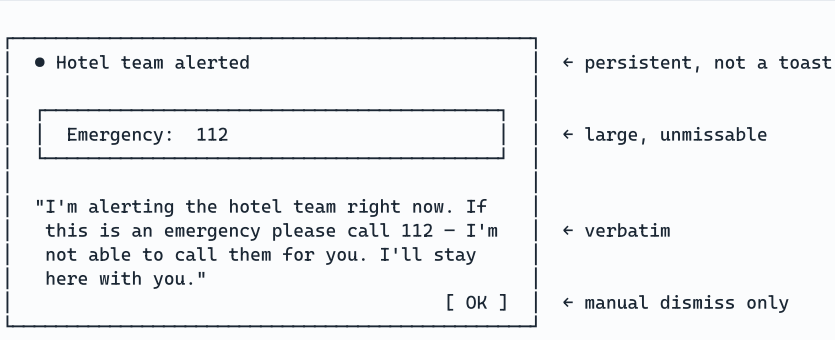
> **WHY "YES" CANNOT BE MISINTERPRETED**
>
> When the guest confirms, the platform executes **the exact action it froze** — it does not re-read their words. So even if someone said "yes, and make it ten sandwiches", the ten does not happen. Useful to know when designing the copy: the confirm step is a yes/no gate, not a place to amend the order.

7. Escalation — the other critical one

> **DO NOT EDIT, SHORTEN, SUMMARISE OR TRANSLATE THIS WORDING**
>
> Escalation text is a **compliance artefact**, not copy. It states plainly what the system *cannot* do — because a guest who believes an ambulance is coming when it is not is the worst outcome this product can produce.
>
> Render `speech` and `screen` verbatim. Do not add your own reassurance on top, and do not replace "I'm not able to call them for you" with something softer.

Requirement	Detail
Persistent	**Never auto-dismiss.** Requires explicit user action to clear.
Prominent	Highest-priority element. It outranks anything else on screen.
Emergency number legible	Large. It arrives in `screen` and comes from property config — it is not always 112.
Verbatim	No paraphrasing, no truncation, no ellipsis.
Stays interactive	The guest can keep talking. Do not lock the interface.

SUGGESTED LAYOUT — ESCALATION



8. Refusals and limits

Refusals are frequent and intentional. Treated as errors, the interface will feel broken when it is working exactly as designed.

Guest asks for	What happens	Design as
A temperature outside the hotel's band	`DENY` , `R6.out_of_bounds`	Neutral information
Something their room type doesn't have	No tool exists; a plain spoken answer	Neutral information
Another guest's details	Refusal — structurally impossible to answer	Neutral. **Never** "access denied" framing
The door unlocked	Refusal; no such capability exists at any level	Neutral, with the front desk offered
Too many changes too fast	`DENY` , rate limit	Gentle, temporary
An order over the spend cap	`DENY` , spend rule	Neutral, offer the front desk

> **TONE**
>
> The service already words refusals well — "I can only help with your own room." Your job is not to improve on it but to **not wrap it in alarming chrome**. No red, no warning triangle, no "Error" heading. It is an answer, not a fault.

9. Degraded and offline

Level	What happened	Guest sees
0	Normal	Nothing
1	AI slow	Nothing — handled invisibly
2	AI unavailable	Simple requests still work; reply names what still works
3	Speech unavailable	**Screen becomes the primary interface**
4	Internet down	Lights, temperature, DND still work locally
5	Everything down	Honest message pointing at the room phone

> **THE PATTERN TO FOLLOW THROUGHOUT**
>
> **Say what still works, not what is broken.** "I can help with lights, temperature and do-not-disturb right now" keeps the guest using the product. "Sorry, I'm having trouble" teaches them to stop trying. The backend already words its degraded replies this way — match it in your own offline and error states.

PART III

The device screen

10. How it works

The in-room hub has a display. Its **content is decided by the server** and arrives as a `screen` message; its **presentation is yours**. The device renders, it never decides.

```
← {"type":"speak",  "text":"I've set it to 22.5 degrees.", "turnId":"turn_…"}
← {"type":"screen", "text":"22.5°C"}
```

You are designing the screen states; the Android team implements them. Two constraints from the protocol: `screen` is **plain text** that may contain `\n` — never markdown, never HTML — and it is only sent when the display carries something the audio does not.

11. Screen states



State	Must communicate	Design notes
IDLE	Room is ready; device is not listening	Ambient. Time, room number, maybe weather. Should be pleasant to have on a bedside table at 3 a.m. — dim, no motion.
LISTENING	**Unambiguously** that the mic is live	The most important honesty signal in the product. Distinct, immediate, impossible to miss.
THINKING	Working on it	May last 30 ms. Fade in after ~150 ms so fast turns don't flash.
SPEAKING	What was said and what changed	Show `screen` if present; otherwise the `speak` text.
CONFIRMING	**The total**	The amount is the largest element. See §6.
MUTED	Mic is off *in hardware*	Unmistakable and persistent. Guests must be able to trust it at a glance from across the room.
OFFLINE	Reduced capability + what still works	Badge, not a takeover.
SERVICE	Needs staff attention	Neutral. Not the guest's problem to solve.

> **TWO STATES CARRY TRUST, NOT JUST INFORMATION**
>
> **LISTENING** and **MUTED** are the two that decide whether a guest believes the device isn't spying on them. If either is subtle, ambiguous, or looks similar to a neighbouring state, the product has a credibility problem no amount of privacy policy fixes.
>
> Design them to be readable from the bed, in the dark, without glasses.

12. Latency and the filled pause

There is a mechanism you should design around rather than fight.

```
guest stops speaking          at    0ms
ack   "Let me sort that—"     at    2ms   ← device speaks HERE
speak "I've set it to 22.5"   at 1846ms
```

The device says something reassuring within milliseconds, from cached audio, then the real answer arrives up to two seconds later. Measured on the real stack.

Implication for the screen	What to do
Audio and screen can be ~2 s apart	Do not wait for `speak` to leave IDLE. Enter THINKING when the utterance ends.
Fast turns are genuinely fast	A spinner that appears for 39 ms is worse than none. Delay it ~150 ms.
The device is already talking during THINKING	THINKING should not look like silence — it should look like attentiveness.

PART IV

Craft

13. Accessibility

VOICE MUST NEVER BE THE ONLY PATH

Guests who are deaf, hard of hearing, non-verbal, or who have atypical speech must be able to reach **every** function through the screen. This is an obligation under ADA and EN 301 549 — and in an emergency the screen works when speech does not, which is exactly when it matters most.

Requirement	Detail
Every spoken reply on screen	Not a summary. The same words.
Every function touch-reachable	If it can be asked for, it can be tapped. No voice-only capability.
Contrast	WCAG AA minimum; AA-large for anything read from across a room.
Text scaling	Support system scaling. In-room devices are used by older guests without their glasses.
Motion	Respect reduced-motion. Nothing that pulses in a dark bedroom.
Touch targets	44 pt minimum. Guests are often standing, holding something, half asleep.
Slow-speech mode	A per-room setting staff enable at check-in extends how long the device waits before deciding the guest has finished. Surface it in the admin UI (§16).
Never colour alone	Especially for MUTED and LISTENING.

14. Copy and tone

Most guest-facing wording comes from the server, already toned per property. Your copy is the scaffolding around it: labels, buttons, empty states, errors.

Principle	Do	Don't
Never claim an outcome you don't have	"I've asked for that"	"Done!" before confirmation
Refusals are information	"That's not something I can do here."	"Error: permission denied"
Say what still works	"I can still do lights and temperature."	"Service unavailable"
Money is explicit	"₹1,240 charged to your room"	"Confirm order"
No jargon	"the front desk"	"escalate to T2 support"
Brief	One idea per screen	Paragraphs on a bedside display

NEVER SURFACE THESE INTERNALS TO A GUEST

`lane`, `tier`, `ruleId`, `turnId`, `verdict`, tool names like `climate.set_temperature`, and `degradationLevel` as a number.

They are exactly right for a debug view or the staff console. To a guest they are noise, and "T3 CONFIRM R11.confirmation_required" is not a sentence anyone should read on holiday.

15. What never to show

Never display	Why
Another room's number or any other guest's details	The service refuses these structurally, but a UI must not reintroduce them by, say, showing a property-wide feed on a guest surface.
Full card numbers, ever	The system blocks them on every path. Never add a field that accepts one — a guest reading a card number aloud in a hotel room is a bad idea on its own terms.
Raw internal errors or stack traces	—
Staff names or internal assignment detail	On guest surfaces.
"AI" framing on an emergency screen	In an escalation the guest needs a phone number, not a disclaimer.

PART V

Staff & admin — the contract

THIS API DOES NOT EXIST YET

Everything in this Part is a **proposed contract**, not something you can call. It is written so you can design screens and mock data now, and so that when it is built it matches what you designed. Roughly a week of work once the decisions in §24 are made.

16. Screens needed

Screen	Who uses it	Must do
Request queue	Housekeeping, maintenance	Live list of open requests with room, category, age, SLA countdown. Assign, start, complete. This is the screen someone stares at all shift — it deserves the most attention.
Escalations	Duty manager, front desk	Unacknowledged emergencies, loudly. Acknowledge with a name attached. **An escalation nobody acknowledges is the worst failure in the system** — design the alarm for that case first.
Room view	Front desk	One room: current state, open requests, orders, recent activity. What you open when a guest phones down.
Activity log	Duty manager, support	What the assistant did and *why* — the record that resolves a billing dispute. Searchable by room and time.
Property config	Ops / onboarding	Room types, capabilities, temperature bands, quiet hours, spend caps. This is what makes a new hotel a config change rather than an engineering project.
Devices	Ops, hotel engineering	Online/offline, signal strength, firmware, room binding. Which rooms have poor Wi-Fi.
Kill switches	Ops	Disable one capability across a property in seconds; put a property in read-only mode. Used during incidents — must be fast and unambiguous.
Menu & knowledge	F&B, front office	Menu items with prices, availability, spoken aliases. Property FAQ.

BEFORE DESIGNING THE REQUEST QUEUE, SETTLE ONE PRODUCT QUESTION

Most hotels already run HotSOS, Quore or Alice, and **no hotel wants a second queue for their staff to watch**. If we push requests into their existing system, this screen becomes a thin monitoring view rather than a working queue — a very different design.

Worth resolving with sales before investing in it.

17. Proposed API contract

Shapes to design and mock against.

17.1 Requests

```
GET  /v1/staff/requests?propertyId=&status=open&category=
  → { "requests": [ {
        "id": "sr_ms71…", "roomNumber": "412", "category": "maintenance",
        "description": "shower dripping", "priority": "normal",
        "status": "open", "createdAt": "2026-07-29T23:14:02Z",
        "slaDueAt": "2026-07-29T23:59:02Z",
        "deferred": true, "dispatchAt": "07:00",
        "assignee": null
      } ] }

POST /v1/staff/requests/{id}/assign     { "assignee": "staff_42" }
POST /v1/staff/requests/{id}/status     { "status": "in_progress" | "completed" }
```

`deferred` and `dispatchAt` matter for the UI: a non-urgent request raised at 23:14 is deliberately queued for 07:00 so the guest isn't disturbed. Show that clearly — otherwise staff see an old untouched ticket and think the system is broken.

17.2 Escalations

```
GET  /v1/staff/escalations?acknowledged=false
  → { "escalations": [ {
        "id": "esc_ms71…", "roomNumber": "412",
        "category": "medical",          // medical | fire | security | welfare | other
        "severity": "critical",
        "summary": "Guest reported chest pain",
        "raisedAt": "2026-07-29T23:14:02Z",
        "requiresAckWithinS": 120,
        "acknowledgedAt": null, "acknowledgedBy": null
      } ] }

POST /v1/staff/escalations/{id}/ack     { "staffId": "staff_42" }
```

17.3 Rooms and devices

```
GET  /v1/staff/rooms/{roomId}
  → { "roomNumber": "412", "roomType": "Deluxe King",
      "state": { "zones": [ { "zone":"bedroom",
                              "lighting": { "on": true, "brightness": 70 },
                              "climate":  { "currentC": 24.5, "setpointC": 22.5,
                                            "minC": 18, "maxC": 26 } } ],
                 "dnd": "off", "freshness": "fresh" },
      "openRequests": [ … ], "recentOrders": [ … ],
      "device": { "deviceId":"d_412a", "online": true, "rssi": -58,
                  "firmware": "2.4.1", "lastSeen": "…" } }
```

`freshness` is worth surfacing: `fresh` / `stale` / `unknown`. On `unknown` the room state could not be read — show it as unknown rather than displaying a stale number as current.

17.4 Activity log

```
GET  /v1/staff/turns?roomId=&from=&to=
  → { "turns": [ {
        "turnId": "turn_ms71…", "at": "…", "roomNumber": "412",
        "transcript": "it's freezing in here",     // redacted
        "response": "I've set it to 22.5 degrees.",
        "outcome": "success", "escalated": false,
        "actions": [ { "tool": "climate.set_temperature", "verdict": "ALLOW",
                       "ruleId": "allow", "result": "applied" } ]
      } ] }
```

This is the dispute-resolution screen. Design it so a duty manager can answer "why did the room get cold at 2 a.m." in under a minute. Transcripts are redacted and retained ~30 days; action records are kept far longer, so a search may return actions with no transcript — handle that gracefully rather than showing an empty row.

17.5 Configuration and kill switches

```
GET  /v1/admin/properties/{id}                    // full config
PUT  /v1/admin/properties/{id}/policy             // temp bands, quiet hours, spend caps
PUT  /v1/admin/properties/{id}/room-types/{rtId}  // capability manifest
PUT  /v1/admin/rooms/{roomId}/overrides           // e.g. disable curtains: motor broken

POST /v1/admin/properties/{id}/kill-switch
     { "disabledTools": ["fnb.place_order"] }     // disable one capability
     { "tierCeiling": "T0" }                      // read-only mode
```

> **KILL SWITCHES ARE INCIDENT TOOLS — DESIGN THEM FOR A BAD NIGHT**
>
> Someone reaches for these when something is going wrong, possibly at 2 a.m., possibly while a manager is on the phone. Requirements: unmistakable current state, obvious blast radius ("this affects all 300 rooms at Whitefield"), a confirmation step, and an equally obvious way to undo. Never bury them in a settings tab.

18. Live updates

Polling a request queue is the wrong shape — a housekeeping tablet should buzz within seconds. Proposed: Server-Sent Events.

```
GET /v1/staff/events?propertyId=p_blr_whitefield
  Accept: text/event-stream

event: service_request.created
data: { "id":"sr_…","roomNumber":"412","category":"maintenance","priority":"normal" }

event: escalation.raised
data: { "id":"esc_…","roomNumber":"412","category":"medical","severity":"critical" }

event: device.offline
data: { "deviceId":"d_412a","roomNumber":"412" }
```

Design for reconnection: SSE drops. On reconnect, refetch the list rather than assuming the stream filled every gap.

19. Permissions model

The staff API will be the **only** inbound path into the system, so it carries the most scrutiny. Roles shape your navigation, so they are worth agreeing early.

Role	Scope	Can
Housekeeping / maintenance	One property	See and update requests assigned to their department
Front desk	One property	All requests, room view, acknowledge escalations
Duty manager	One property	Everything above + activity log + kill switches
Ops	Many properties	Configuration, devices, onboarding
Support	Many, audited	Read-only activity log

Every request will be validated against the caller's property claim server-side. Design as if a user may hold access to several properties: a property switcher is easier to add now than to retrofit.

PART VI

Practical

20. Running it locally

```
npm install
npm run gateway        # http://localhost:8443

curl -X POST http://localhost:8443/v1/turn \
  -H 'content-type: application/json' \
  -d '{"deviceId":"d_412a","transcript":"turn on the lights"}'
```

No API key, no database, no internet. Three seeded devices:

deviceId	Room	Type	Good for testing
`d_412a`	412	Deluxe King	Missing capabilities — no curtains, scenes or music
`d_511a`	511	Suite	Everything available
`d_512a`	512	Suite	Capability disabled by config

21. Test scenarios

on `d_412a` with `asrConfidence: 0.4`

Send	State to exercise	Watch for
`turn on the lights`	Fast success	Spinner shouldn't flash at 39 ms
`it's freezing in here`	Normal success	Reads state, sets 22.5 °C
`close the curtains and make it warmer`	Two actions	Two entries in `calls`
`set the temperature to 35 degrees`	**Refusal**	200 OK. No error chrome.
`I'd like a club sandwich and a beer`	**Confirmation**	Total prominent; no auto-confirm
`yes`	Confirmed	State clears
`no`	Declined	Nothing charged
`I'm having chest pain`	**Escalation**	Persistent, verbatim, number legible
`who is staying in room 402`	Privacy refusal	Neutral tone
`close the curtains`	Absent capability	Plain answer, no failure UI
`mumble`	Clarify	Empty `calls`; invite a retry

22. Design checklist

Area	Check
Confirmation	Total is the largest element · never auto-confirms · visible cancel · expires at 90 s
Escalation	Verbatim · persistent · manual dismiss only · emergency number large
Refusals	Neutral, not error chrome · never "access denied"
Loading	Delayed ~150 ms so fast turns don't flash
Offline / degraded	Says what still works, not what is broken
Accessibility	Every spoken reply on screen · every function touch-reachable · AA contrast · text scaling · reduced motion
Privacy	LISTENING and MUTED unmistakable from across a dark room
Never shown	Internals, other rooms, card fields, stack traces
Staff screens	SLA countdowns · deferred requests explained · unacknowledged escalations alarm · kill-switch blast radius stated

23. Reference data

Menu (seeded)

ID	Name	Price	Spoken as	Available
`m_club`	Club Sandwich	850 INR	sandwich, club	yes
`m_dosa`	Masala Dosa	450 INR	dosa, south indian	yes
`m_beer`	Kingfisher Premium	390 INR	beer, lager	yes
`m_caesar`	Caesar Salad	620 INR	salad	**no**

Aliases exist because a guest ordering "a beer" must not fail because the menu calls it "Kingfisher Premium". If you build menu-management UI, that field needs a first-class editor.

Property policy (seeded)

Temperature band	18–26 °C
Quiet hours	22:00–07:00, non-urgent requests deferred to morning
Spend caps	6,000 INR per order · 15,000 INR per stay
Open requests	10 per stay, 3 per category
Emergency number	112 (per property — do not hardcode)

24. Decisions needed from you

The staff API is blocked on two answers. Everything else is ready.

Decision	Options	Why it matters
API style	REST or GraphQL	Changes the shape enough that guessing wastes a week. REST is simpler; GraphQL suits a dashboard pulling many entities at once.
Identity provider	Cognito · Auth0 · Google Workspace · something in use	Determines the login flow you design and how property scoping is carried.
Ticket ownership (product, not blocking)	Woohoo owns the queue, or pushes into HotSOS/Quore	Decides whether the request queue is a working tool or a monitoring view.

If your sprint has already started, an OpenAPI spec can be produced first so you mock against the contract while the implementation lands behind it.
